

Apuntes para entender y defender el proyecto

Guía de lectura del código - Sistema de Asistencias Digital con Reconocimiento Facial

Apuntes para entender (y defender) el proyecto -- de 0 a 100

Como usar estos apuntes. Están pensados para leerse **con el código abierto al lado**. Cada sección te dice *que* archivo abrir y *que* mirar en él. No intentes memorizar: sigue los recorridos, y cuando entiendas *por que* cada pieza está donde está, la defensa sale sola. Orden sugerido de lectura: 1 -> 2 -> 3 -> 4 (los recorridos) -> 5 (los mecanismos) -> 6 (autoevaluación).

1. Que problema resuelve (el "para que")

Una institución con 200-400 docentes registra la asistencia en papel y firma. Eso es lento de auditar, fácil de falsear y casi imposible de reportar. Este sistema lo reemplaza: una **camara fija en secretaria** reconoce al docente y registra su asistencia automáticamente.

Los tres hechos del dominio que explican casi todas las decisiones técnicas:

1. **La cámara está en secretaria, no en el aula.** El sistema no "ve" la clase: la *deduce* cruzando la hora del registro con los horarios cargados. De ahí salen el RF-18 (que materia es) y toda la lógica de ventanas horarias.
2. **El docente es sujeto pasivo.** No se loguea, no usa la interfaz, solo se para frente a la cámara. Por eso Docente y Usuario son **entidades distintas**: el docente no tiene contraseña ni rol.
3. **Trabaja con datos biométricos = datos sensibles.** Ley 25.326 + Resolución AAIP 255/2022. De ahí sale todo el módulo de consentimiento, el cifrado, el no-guardar-fotos y el borrado físico ante supresión.

El criterio rector de todo el diseño, si te preguntan una sola cosa: *ante duda, el sistema NO registra*. Un falso positivo (confundir a un docente con otro) mete un dato falso en un registro legal; un falso negativo solo obliga a una carga manual. Por eso el umbral es estricto y la carga manual es la red de seguridad, no un parche.

2. El modelo de datos (el "de que habla")

Abri: `src/main/java/edu/cent35/asistencias/model/`

Son 22 archivos. Léelos en este orden, que es el orden en que se entiende el negocio:

2.1 El tenant y quien opera

Archivo	Que es
<code>Institucion.java</code>	La raíz. Cada institución es un tenant (inquilino) del sistema.
<code>BaseTenantEntity.java</code>	Superclase con la columna <code>institucion_id</code> . Toda entidad que la extienda queda aislada por institución.
<code>Usuario.java</code> + <code>Rol.java</code> + <code>RolCodigo.java</code>	Quien se loguea. Dos roles: <code>INSTITUCION</code> (cuenta raíz) y <code>ADMIN</code> (operativo).

2.2 La estructura academica (jerarquia de 4 niveles)

```
Carrera  -> Materia  -> Comision  -> Horario
          (titular)  (docente   (dia, hora inicio/fin,
                      asignado)   tolerancia_min)
```

Archivo	Que mirar
Carrera.java	Simple. Agrupa materias.
Materia.java	Tiene carrera y docenteTitular (responsable academico).
Comision.java	Tiene materia y docenteAsignado (el que efectivamente dicta y a quien se le toma asistencia). Ojo: no extiende BaseTenantEntity -- su tenant lo determina la materia.
Horario.java	La franja: diaSemana, horaInicio, horaFin y toleranciaMin. Ese ultimo campo es el que decide Presente vs Tarde.

Pregunta tipica del tribunal: "El RF-14 dice que el horario tiene docente, pero en tu modelo el docente esta en la comision. ¿Por que?" **Respuesta:** normalizacion. Una comision = un docente; la comision tiene varios horarios (lunes y miercoles, por ejemplo). Poner el docente en cada horario permitiria inconsistencias (que el lunes lo de uno y el miercoles otro dentro de la misma comision).

2.3 El docente y su biometria

Archivo	Que mirar
Docente.java	Datos personales. No tiene contrasenia: no se loguea.
ConsentimientoBiometrico.java	Historico (N por docente). Guarda version del texto firmado, metodo, fechas y auditoria forense: IP y navegador de quien lo otorgo y de quien lo revoco.
EstadoConsentimiento.java	NUNCA_OTORGADO / ACTIVO / REVOCADO. Se calcula a partir del registro mas reciente.
ModeloFacial.java	El dato biometrico. Campo embeddingCifrado (BLOB). N por docente, 1 activo .

2.4 La asistencia

Archivo	Que mirar
Asistencia.java	El nucleo. Mira dos cosas: el UNIQUE (docente, horario, fecha) (garantiza idempotencia) y el campo confianza.
EstadoAsistencia.java	PRESENTE / TARDE / AUSENTE.
MetodoAsistencia.java	AUTOMATICO (lo genera el sistema) / MANUAL (lo cargo una persona).
AsistenciaManual.java + MotivoCargaManual.java	Detalle 1:1 cuando la carga fue manual: quien, con que motivo del catalogo, y detalle libre.
JustificacionAusencia.java	Detalle 1:1 sobre una ausencia justificada.

2.5 El archivo mas facil de pasar por alto

model/package-info.java -- parece vacio pero declara el @FilterDef("tenant") de Hibernate, o sea **la definicion del filtro multi-tenant de todo el sistema**. Si ese archivo desaparece, la app no arranca.

3. La arquitectura (el "como esta armado")

3.1 Organizacion: package-by-layer

Abri: src/main/java/edu/cent35/asistencias/

```

controller/ (15) -> recibe HTTP, valida forma, delega. NO tiene logica de negocio.
service/    (19) -> la logica de negocio + las reglas + las transacciones.
repository/ (14) -> acceso a datos (Spring Data JPA).
model/      (22) -> entidades JPA + enums.
dto/        (29) -> objetos de transporte entre la UI y el controller.
config/     (9)  -> seguridad, multi-tenancy, scheduling, web/JPA.

```

Regla de lectura: una operacion **siempre** viaja controller -> service -> repository -> model. Si buscas "donde esta la regla de negocio X", esta en service/. Si buscas "que URL hace X", esta en controller/.

Contexto para la defensa: el proyecto arranco con package-by-feature (ADR-0001) y se migro a package-by-layer (ADR-0006) a pedido del docente. Ambas son validas; sabe nombrar el trade-off: por-capa es mas reconocible y directo; por-feature agrupa mejor cada dominio. **Y sabe contar que esa migracion tuvo un costo real:** rompio el pointcut del aspecto multi-tenant (TD-007), que se detecto y corrigio despues.

3.2 Los cuatro mecanismos transversales

Abri: src/main/java/edu/cent35/asistencias/config/

Archivo	Que hace
SecurityConfig.java	Login por formulario, rutas publicas vs protegidas, BCrypt, logout.
CustomUserDetails.java / CustomUserDetailsService.java	El "principal" de Spring Security, extendido con institucionId -- esa es la pieza que conecta autenticacion con multi-tenancy.
TenantContext.java	ThreadLocal con el id de la institucion del request actual.
TenantInterceptor.java	Lo llena al inicio de cada request y lo limpia al final (obligatorio: los hilos se reutilizan).
TenantFilterAspect.java	Activa el filtro Hibernate en cada @Service. Lee el javadoc completo: cuenta la historia de TD-007.
PlanificacionConfig.java	Habilita @Scheduled (el job de ausencias).
WebMvcConfig.java / JpaConfig.java	Registro del interceptor y config de JPA.

4. Recorridos de codigo (seguí el hilo con los archivos abiertos)

Esta es la parte mas util. Cada recorrido es un flujo real de punta a punta.

Recorrido 1 -- Login y activacion del tenant

Por que empezar aca: todo lo demas depende de que el tenant este bien cargado.

1. controller/LoginController (o la config de form login en SecurityConfig) recibe usuario y contrasenia.
2. config/CustomUserDetailsService busca el Usuario en la BD y lo envuelve en un CustomUserDetails -- **que ademas del username lleva el institucionId.**
3. En **cada request posterior**, config/TenantInterceptor.preHandle() lee ese principal y hace TenantContext.set(institucionId). Ademas escribe tenantId y userId en el MDC, por eso los logs muestran de que institucion es cada linea.
4. Cuando el request termina, afterCompletion() hace TenantContext.clear().
5. Mientras tanto, config/TenantFilterAspect activa el filtro de Hibernate en cada metodo de @Service, de modo que las queries agregan solas WHERE institucion_id = :institucionId.

Lo que tenes que poder decir: "el aislamiento no depende de que yo me acuerde de filtrar: se aplica automaticamente en la capa de datos".

Recorrido 2 -- Registro del rostro de un docente

Abri: controller/RegistroFacialController -> service/ModeloFacialService -> service/DeteccionRostroService
-> service/MotorLbphService -> service/CifradoBiometricoService

1. **Pantalla** (GET /docentes/{id}/rostro/registro): el controller verifica que el docente este activo y que **tenga consentimiento ACTIVO**. Sin eso, no deja registrar. *(Esta es la regla legal materializada en codigo: mirala en el controller.)*
2. **El navegador** (static/js/registro-facial.js) graba ~30 s y manda los frames en base64 por JSON.
3. ModeloFacialService.registrar() -- **el metodo central**, leelo entero:
 - Revalida consentimiento (defensa en profundidad: la UI ya lo valido, el service lo valida de nuevo).
 - Por cada frame llama a DeteccionRostroService.extraerRostroNormalizado(), que usa el **Haar Cascade** para recortar la cara, pasarla a gris y escalarla a 200x200. Los frames sin cara clara **se descartan**.
 - Si quedan menos frames validos que el minimo configurado, falla con mensaje claro.
 - MotorLbphService.entrenar() entrena el modelo LBPH y lo **comprime con gzip**.
 - CifradoBiometricoService.cifrar() lo cifra con **AES-256-GCM**.
 - Da de **baja logica** el modelo anterior (RF-09) y guarda el nuevo.
4. **Las imagenes nunca se persisten**: viven en memoria y se cierran en el finally.

Recorrido 3 -- Pase de asistencia (EL flujo estrella de la demo)

Abri: controller/PaseAsistenciaController -> service/PaseAsistenciaService -> service/IdentificacionFacialService -> service/AsistenciaService

El navegador (static/js/pase-asistencia.js) manda **un frame por segundo**. Cada frame recorre esto:

Paso A -- Identificar (IdentificacionFacialService.identificar()):

1. Detecta y recorta el rostro (mismo servicio que en el registro: ****la misma normalizacion en registro y en consulta****, si no, no comparan bien).
2. Trae los modelos activos del tenant y los tiene en un **cache en memoria** (descifrar + descomprimir + deserializar en cada frame seria inviable).
3. Hace predict() contra cada uno y se queda con la **menor distancia**.
4. Si distancia > umbral -> devuelve **no reconocido y no marca nada**.
5. Loguea la linea CALIBRACION ... (de ahi salen los numeros del protocolo de calibracion).

Paso B -- Marcar (AsistenciaService.marcarAutomatica()):

1. Busca los horarios de hoy del docente.
2. elegirHorarioEnCurso() -- **leelo con atencion, es el RF-18**: filtra los horarios cuya ventana [horaInicio - tolerancia, horaFin] contiene el momento actual, y si hay mas de uno aplica el desempate de tres niveles (sin marca previa -> inicio mas cercano -> menor id).
3. Si no hay ninguno -> sinClase (y la UI lo muestra en amarillo).
4. Si ya hay marca para ese (docente, horario, fecha) -> yaEstaba (azul). **Idempotencia**.
5. Si no -> calcula PRESENTE o TARDE segun si llego antes o despues de horaInicio, convierte la distancia a un score 0-1 y guarda con saveAndFlush, atrapando la violacion de UNIQUE por si hubo carrera.

Paso C -- Responder: `PaseAsistenciaService` arma el DTO combinado y el JS pinta el recuadro (verde = marcado, azul = ya estaba, amarillo = sin clase, rojo = no reconocido) y **pausa 5 segundos**.

Recorrido 4 -- El cierre del día (ausencias)

Abri: `service/GeneradorAusenciasService`

1. `@Scheduled` lo dispara segun el cron configurado.
2. **Como no hay request, no hay tenant:** el job itera las instituciones activas y hace `TenantContext.set(...)` / `clear()` **manualmente** por cada una. Mira el `try/finally`.
3. Por cada horario del día ya terminado sin marca, crea la fila `AUSENTE`.
4. Si el docente marca justo en ese instante, el `UNIQUE` de la BD resuelve la carrera a favor de la marca real y el job **descarta sin romperse**.

Recorrido 5 -- Consultas y reportes

Abri: `controller/AsistenciaController` + `service/AsistenciaService.listarDelDia()` + `controller/ReporteController` + `service/ReporteAsistenciaService`

- `listarDelDia()` implementa el **modelo hibrido**: trae las asistencias reales y ****suma** filas `AUSENTE` virtuales** para los horarios terminados sin marca (ventana de gracia hasta que corra el job).
- `ReporteController.descargarCsv()` escribe el CSV **con BOM UTF-8** (para que Excel respete los acentos) y separador `;`.

5. Los cinco mecanismos que tenes que dominar

Si entiendes estos cinco, entiendes el proyecto.

5.1 Multi-tenancy en tres capas

Capa	Donde	Que cubre
1. Filtro Hibernate automatico	<code>model/package-info.java</code> + <code>config/TenantFilterAspect</code>	Queries derivadas y <code>findAll()</code> sobre entidades <code>tenant-scoped</code>
2. <code>WHERE ... = :tenantId</code> explicito	Las <code>@Query</code> de <code>repository/</code>	Queries con JOIN -- el filtro NO se propaga a JOINS (TD-003)
3. Validacion en service	Metodos <code>obtenerXValidado(...)</code>	Accesos por id; responde "no encontrado" para no revelar que el dato existe en otra institucion

Las dos anécdotas que valen oro en la defensa:

- **TD-003:** descubriste en pruebas que el filtro no se propaga a los JOINS de JPQL y un `tenant` veia comisiones de otro. Lo corregiste y lo documentaste.

- **TD-007:** la reorganizacion de paquetes dejo el aspecto **silenciosamente inactivo**; lo detectaste en la revision final, lo corregiste cambiando el pointcut a uno basado en anotacion, y **agregaste un test que se pone rojo si vuelve a pasar**.

Poder contar dos bugs propios, con causa, solucion y prevencion, demuestra mas dominio que decir "no tuve problemas".

5.2 Privacidad biometrica

Tres reglas, cada una con su archivo:

- **No se guardan fotos** (RNF-08): los `Mat` de OpenCV se cierran en `finally`; no hay tabla ni carpeta de imagenes.
- **El vector va cifrado** (RNF-07): `CifradoBiometricoService`, AES-256-GCM con clave derivada por PBKDF2.
- **Supresion = borrado fisico** (RNF-14): `ModeloFacialService.suprimirDatosBiometricos()` hace `DELETE` real, evita el cache en memoria, y **conserva las asistencias** porque la FK es `ON DELETE SET NULL`.

Por que el borrado del vector es fisico si todo lo demas es baja logica: porque una fila con `activo=false` **sigue conteniendo el dato sensible**. El derecho de Cancelacion exige que desaparezca. Es la excepcion documentada a la regla general.

5.3 Idempotencia del pase

Tres niveles, y hay que saber nombrarlos:

1. **BD:** `UNIQUE (docente_id, horario_id, fecha)`.
2. **Service:** verifica antes de insertar y atrapa `DataIntegrityViolationException` por si hubo carrera.
3. **UI:** pausa 5 s tras marcar, para no bombardear el servidor.

5.4 Como funciona LBPH (y su limite)

Abri: `service/MotorLbphService`

LBPH entrena **un modelo por docente** y al predecir devuelve una **distancia** (menor = mas parecido). **No es un embedding**: no hay un vector comparable entre personas.

- **Esto contradice el RF-08**, que pedia embeddings reutilizables. Esta **formalizado** como desviacion aceptada en ADR-0007**, con el camino de migracion verificado (`SFace` ya esta disponible en las librerias del proyecto).
- **Nunca digas "embeddings"** describiendo lo que hace hoy.
- El modelo se serializa a YAML, se **comprime con gzip** (el YAML de OpenCV es enorme y repetitivo; sin comprimir rompia el `max_allowed_packet` de MariaDB) y **despues** se cifra.

5.5 `open-in-view=false` y las asociaciones LAZY

Por que importa: el proyecto desactiva `open-in-view`, o sea que la sesion de Hibernate se cierra al terminar el service. Si Thymeleaf intenta leer una asociacion LAZY despues, **explota**. Por eso vas a ver dos patrones:

- `JOIN FETCH` en las `@Query`.
- Metodos `touchLazy(...)` en los services que "tocan" la asociacion dentro de la transaccion.

Ya te paso una vez (un error 500 al volver de otorgar consentimiento). Es una buena anecdotita si preguntan por rendimiento: *"lo desactive a proposito para que los N+1 se hagan visibles en vez de esconderse"*.

6. Autoevaluacion (respondelas en voz alta)

Si podes responder estas doce sin mirar, estas listo:

1. ¿Por que `Docente` y `Usuario` son entidades distintas?
2. ¿Donde vive el aislamiento multi-tenant y cuantas capas tiene? Nombralas.
3. Un admin pide `/docentes/999/editar` donde 999 es de otra institucion: ¿que pasa y por que se responde "no encontrado" y no "prohibido"?

4. ¿Que hace exactamente `elegirHorarioEnCurso` cuando hay dos horarios en ventana?
5. ¿Por que el sistema no registra cuando la distancia supera el umbral?
6. ¿Que se guarda en `modelos_faciales.embedding_cifrado`? (Cuidado: **no** es un embedding.)
7. ¿Por que se comprime con gzip antes de cifrar?
8. ¿Que pasa con las asistencias historicas cuando un docente ejerce el derecho de Cancelacion?
9. ¿Como se marca AUSENTE si el docente nunca aparece? (Nombra las dos capas.)
10. ¿Como propaga el tenant un job `@Scheduled`, si no tiene request?
11. ¿Que garantiza que el pase no duplique la marca si el docente se queda frente a la camara?
12. ¿Cual es la debilidad mas grande del sistema y como la mitigas?

7. Mapa rapido: requerimiento -> archivo

RF/RNF	Donde vive
RF-01/02/03	<code>config/SecurityConfig</code> , <code>config/CustomUserDetailsService</code> , <code>service/UsuarioService</code>
RF-04, RNF-05/10	<code>model/BaseTenantEntity</code> , <code>model/package-info</code> , <code>config/Tenant*</code>
RF-07	<code>service/DocenteService</code>
RF-08/09	<code>service/ModeloFacialService</code> , <code>service/MotorLbphService</code> , <code>service/DeteccionRostroService</code>
RF-10, RNF-13	<code>service/ConsentimientoBiometricoService</code> , <code>service/TextoConsentimiento</code>
RF-11 a RF-14	<code>service/CarreraService</code> , <code>MateriaService</code> , <code>ComisionService</code> , <code>HorarioService</code>
RF-15/16/20	<code>static/js/pase-asistencia.js</code> , <code>service/IdentificacionFacialService</code>
RF-17/18/19/21	<code>service/AsistenciaService</code> (<code>marcarAutomatica</code> , <code>elegirHorarioEnCurso</code>)
RF-19 (ausencias)	<code>service/GeneradorAusenciasService</code>
RF-22 a RF-24	<code>service/AsistenciaService.marcarManual</code> , <code>model/AsistenciaManual</code>
RF-25/26	<code>service/AsistenciaService.justificarAusencia</code>
RF-27 a RF-30	<code>service/ReporteAsistenciaService</code> , <code>controller/ReporteController</code>
RNF-06	<code>config/SecurityConfig</code> (BCrypt)
RNF-07/08/14	<code>service/CifradoBiometricoService</code> , <code>ModeloFacialService.suprimirDatosBiometricos</code>
RNF-19	<code>resources/db/migration/</code> (6 migraciones Flyway)

8. Lo que NO esta hecho (decilo vos primero)

Tema	Estado	Como se defiende
Vivacidad / anti-spoofing	No implementado	Candidato RF-38, con solucion disenada (parpadeo/movimiento dentro de los 3 s). Mitigacion: la camara esta en secretaria, a la vista del personal -- el ataque no es anonimo.
Embeddings (RF-08)	LBPH en su lugar	Desviacion formalizada en ADR-0007, con migracion verificada y planificada.
Export PDF / Excel / graficos (RF-31/32/33)	Hay CSV	Alcance del prototipo; los datos ya estan listos para cualquier formato.
Modo claro (RNF-22)	Solo oscuro	Pendiente menor de UI.

Tema	Estado	Como se defiende
Auditoria en runtime (RF-34/36)	Tabla creada, sin llenar	Estructura lista; el registro automatico quedo fuera del alcance.

La frase que cierra bien una defensa: *"Se exactamente que hace mi sistema, que no hace, y por que tome cada decision. Los limites que tiene estan documentados con su solucion propuesta."*